

Academic Question Answering System Using Retrieval-Augmented Generation

Project Documentation · Project ID P_102

Aman Singh · September 2026

Repository: github.com/not-Lucifer/RAG-based-Question-Answering-System

1. Abstract

Students keep their study material as many PDFs (lecture notes, textbooks and previous-year papers), so finding one precise answer means searching every file by hand. General-purpose chatbots answer instantly but are not grounded in the student's syllabus, cannot say where an answer came from, and may be confidently wrong. This project is a **Retrieval-Augmented Generation (RAG)** system that answers questions **only from the student's own PDFs** and **cites the file and page** of every claim. Uploaded PDFs are extracted page by page, cleaned, split into overlapping chunks, embedded with a local sentence-transformer model and stored in a ChromaDB vector database. For each question the most relevant chunks are retrieved and passed to a large language model, a local LLaMA 3.2 3B served by Ollama or OpenAI's gpt-4o-mini, which answers with inline citations. When nothing relevant is found, the system replies *"I couldn't find this in your uploaded material."* instead of guessing.

The system supports follow-up questions, subject and document filters, three retrieval strategies (similarity, MMR and hybrid BM25 + vector), streaming answers and chat history, through a FastAPI backend and a Streamlit interface that run on an ordinary laptop. On a 30-question evaluation over three real course PDFs (53 pages), it retrieved the correct page in the top five for every answerable question (Hit@5 = 1.00, MRR = 0.95), refused all out-of-scope questions, and answered in about 5.4 s on a laptop GPU. All 102 automated tests pass offline.

2. Tech Stack

Layer	Technology	Purpose
Language	Python 3.13 (compatible with 3.11+)	Backend, pipeline, UI and scripts
Orchestration	LangChain 1.4 (LCEL)	Prompt templates, chains, retriever interfaces
PDF extraction	PyMuPDF 1.28	Page-wise text extraction with 1-based page numbers
Chunking	RecursiveCharacterTextSplitter	1000-character chunks, 150 overlap, never crossing pages
Embeddings	all-MiniLM-L6-v2 (local, CPU); OpenAI text-embedding-3-small (optional)	384-dimension sentence vectors
Vector store	ChromaDB 1.5	Persistent nearest-neighbour search with metadata filters
Keyword search	rank-bm25	Exact-term matching for hybrid retrieval
LLM	Ollama llama3.2:3b (local); OpenAI gpt-4o-mini (cloud)	Answer generation, switched by configuration
Backend API	FastAPI 0.142, Uvicorn, Pydantic v2	REST endpoints, validation, Server-Sent Events streaming
Persistence	SQLite + SQLAlchemy 2.1	Documents, chat sessions, messages, feedback
Frontend	Streamlit 1.64	Upload, chat, citations, filters, sessions
Configuration	pydantic-settings, .env, settings.yaml	Provider switch, tunables; secrets kept out of code
Quality	pytest, FastAPI TestClient, Streamlit AppTest, ruff, black	Offline test suite, linting, formatting

Layer	Technology	Purpose
Deployment	Render Blueprint (render.yaml)	API + UI web services with a persistent disk (prepared)

Key design decisions

- **Local-first:** embeddings and the LLM can run entirely on the laptop (MiniLM on CPU, LLaMA 3.2 3B on GPU); OpenAI is an optional configuration switch.
- **Refusal before generation:** if the best retrieved chunk scores below 0.25 cosine similarity, the system answers "not found" without building or calling the LLM; the grounded prompt is a second line of defence.
- **Index safety:** the embedding model's name is stored with the collection; a different model is refused (INDEX_MISMATCH) until the index is rebuilt, because vectors from different models are incomparable.
- **Security:** API keys live only in a git-ignored .env file, are masked when printed and redacted from logs; uploads are checked for size, type and PDF signature and stored under generated names.

3. Flow and Architecture

The system has two pipelines sharing one persistent vector store: an **ingestion pipeline** that runs when a document is uploaded, and a **query pipeline** that runs for every question.

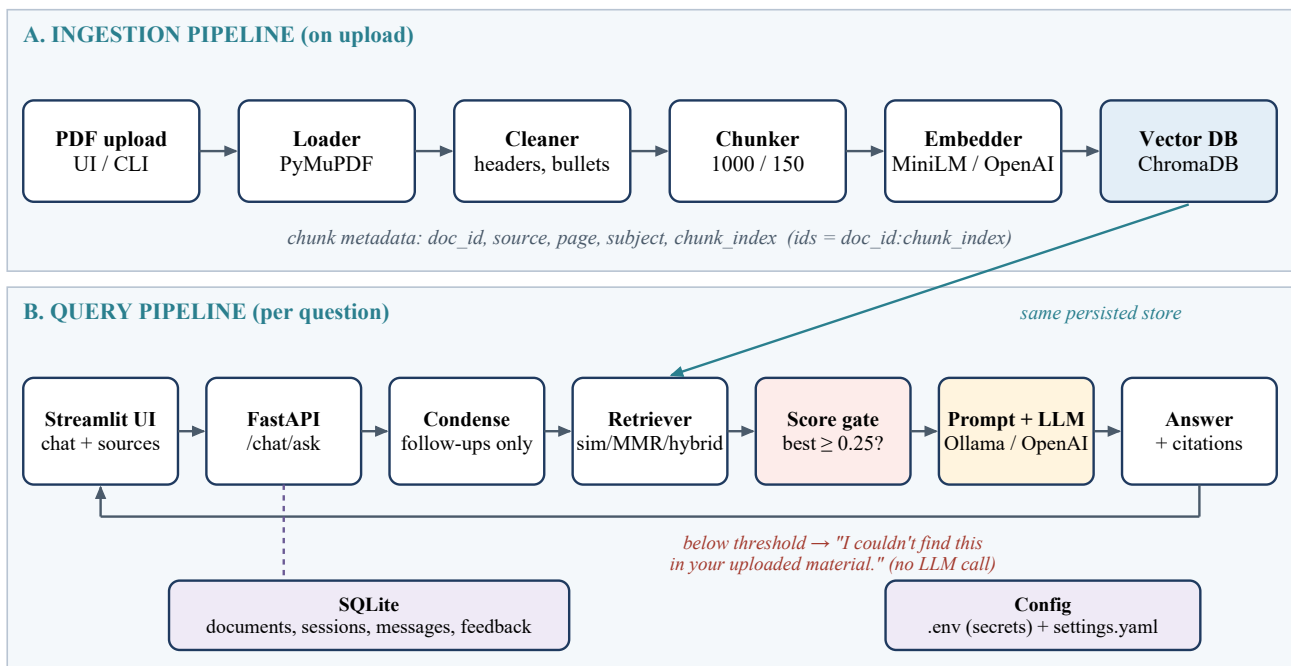


Figure 1. System architecture: ingestion and query pipelines.

Ingestion flow

The upload is validated (size limit, .pdf extension, MIME type, %PDF signature) and de-duplicated by SHA-256 hash, saved as `<uuid>.pdf` and registered in SQLite with status PROCESSING. PyMuPDF extracts one document per page; the cleaner removes running headers and footers (lines repeated on more than half the pages), bare page numbers and hyphenation breaks, re-joins text split one word per line and turns bullet glyphs into list items. Pages are split into overlapping chunks, embedded in batches of 32 and upserted into ChromaDB; the document becomes READY, or FAILED with a readable reason.

Query flow

- The UI sends the question, session id, filters and top-k to **POST /chat/ask** (or /chat/stream).
- The last four turns of the session are loaded; a follow-up such as "what are its types?" is rewritten into a standalone question. Complete questions are used as-is, and a rewrite that drops the question's topic is rejected.

- The retriever returns the top 5 chunks (similarity, MMR over 20 candidates, or hybrid BM25 + vector with weighted reciprocal rank fusion), each with a cosine relevance score; subject and document filters apply.
- Score gate: below 0.25 the system replies "I couldn't find this in your uploaded material." immediately.
- Otherwise the numbered context blocks [1] (file.pdf, p.12) and the question are sent to the LLM, whose system prompt allows answers only from that context, with inline citations.
- The question, answer, sources and latency are stored in SQLite; the UI shows the answer with expandable source cards (file, page, snippet) and feedback buttons.

Layered design

Layer	Code	Responsibility
Presentation	frontend/	Upload, chat, sources, settings; talks to the backend over HTTP only
API	app/api/routes/	Validation, routing, uniform {error, detail} error responses
Service	app/services/	Orchestrates ingestion and chat; owns database transactions
RAG core	app/ingestion, embeddings, vectorstore, retrieval, llm, chains	Pipeline logic; never imports the web framework, so it is testable in isolation
Data	ChromaDB, SQLite, data/raw	Vectors with metadata; documents, chats and feedback; original files

4. Directory Structure

Path	Contents
app/main.py	FastAPI application factory, CORS, error handlers, background model warm-up at startup
app/core/	config.py (settings from .env + settings.yaml), logging.py (secret redaction), exceptions.py (error codes)
app/api/routes/	health.py, documents.py (upload, list, delete, reindex), chat.py (ask, SSE stream), sessions.py (history, feedback)
app/schemas/	Pydantic request and response models
app/ingestion/	loader.py (PyMuPDF), cleaner.py, chunker.py, pipeline.py (load → clean → chunk → store)
app/embeddings/	factory.py: local MiniLM or OpenAI embeddings
app/vectorstore/	store.py: persistent ChromaDB, deterministic chunk ids, embedding-model check
app/retrieval/	retriever.py (similarity, MMR, filters, scores), hybrid.py (BM25 + RRF), reranker.py (optional)
app/llm/	factory.py (Ollama / OpenAI), prompts.py (system, QA and condense prompts)
app/chains/	rag_chain.py (LCEL chain, score gate, streaming), memory.py (history and follow-up rewriting)
app/db/, app/services/	SQLAlchemy engine, models and CRUD; document_service.py and chat_service.py
frontend/	streamlit_app.py, api_client.py, components/ (sidebar, chat_view, source_card)
scripts/	ingest_folder.py, ask_cli.py, reset_index.py, run_eval.py, make_samples.py
evaluation/	qa_dataset.json (30 questions), dataset.py (validation), metrics.py (Hit@k, MRR, LLM judges)
tests/	9 test modules and confest.py with fake embeddings and a fake chat model
config/settings.yaml	Chunking, retrieval, memory and embedding tunables
data/	samples/ (sample notes); raw/, vectorstore/ and app.db are created at run time and git-ignored
docs/	architecture.md, adr/ADR-001.md, report/, screenshots/
Project root	README.md, requirements.txt (+ -prod, -ui, .lock), .env.example, Makefile, make.ps1, pyproject.toml, render.yaml

5. How the Code Was Tested

Testing was done in five stages, from small automated checks of single functions up to using the finished application with real course notes. Every bug found in a later stage was fixed and then covered by a new automated test, so it cannot silently return.

Stage	What was done	How
1. Automated tests while building	Each module was tested as it was written: PDF loading and cleaning, chunking, retrieval modes and filters, the RAG chain, every API endpoint and the Streamlit UI. The suite was re-run after every change.	pytest with a fake embedding model and a fake chat model, so tests run offline in about 30 s; ruff and black for code quality
2. API testing	The backend was started and each endpoint was called directly: health check, PDF upload, duplicate upload (409), invalid file (400), questions inside and outside the notes.	Swagger UI at /docs and curl; a separate install from requirements-prod.txt confirmed the server starts without PyTorch
3. Manual end-to-end testing	Three real course PDFs were uploaded and the app was used like a student: in-syllabus questions (resume vs CV, types of interviews), out-of-syllabus questions (3NF, IPL 2020), follow-ups ("what are its types?"), subject filters, chat switching, feedback and streaming.	Streamlit UI with Ollama (llama3.2:3b); screenshots taken for the report
4. Performance measurement	Time per step was measured for cold and warm starts, normal questions, follow-ups and refusals, before and after optimisation.	Timing scripts against the running model; Ollama GPU statistics
5. Quality evaluation	A 30-question test set with expected pages measured retrieval (Hit@5, MRR), refusals and answer quality, comparing similarity, MMR and hybrid search at two chunk sizes.	scripts/run_eval.py; every generated answer saved for review

Bugs found by testing and fixed

Problem found	Found in	Fix
Off-topic questions failed with "LLM unavailable" when no LLM was configured	Stage 2	The LLM is now created only after the relevance check passes
"what is resume?" asked after "what is 3nf" was rewritten by the small model into a 3NF question and refused	Stage 3	Only real follow-ups are rewritten; rewrites that lose the topic are rejected
An answer disappeared when the page was used while it was still loading	Stage 3 (screenshot)	The UI keeps an answer that already arrived, or says the question was interrupted
First question after start-up took about 38 s	Stage 4	Models are preloaded in the background; Ollama keeps the model in memory for 30 min
The answer-quality judge scored every answer 1 out of 5	Stage 5	Judge now lists each claim before scoring; checked against a deliberately wrong answer
Log filter hid the word "Bearer" but left the secret token visible	Stage 1	Separate redaction rules for tokens and key=value pairs

Commands to repeat the tests

Command (Windows PowerShell)	Purpose
<code>.\make.ps1 test</code>	Run all 102 automated tests (offline)
<code>.\make.ps1 lint</code>	Code style and quality checks (ruff, black)
<code>.\make.ps1 api</code> and <code>.\make.ps1 ui</code>	Start the backend (localhost:8000/docs) and the UI (localhost:8501) for manual testing
<code>python scripts\run_eval.py --llm</code>	Retrieval and answer-quality evaluation on the uploaded notes

6. Results and Performance

Automated test coverage

The suite contains **102 tests** that run fully offline in about 30 seconds: a deterministic fake embedding model, a fake chat model and temporary directories mean no model downloads, no API calls and no changes to real data. Linting (ruff) and formatting (black) are clean.

Area	Tests	What is verified
Configuration, logging	7	Settings precedence, secret masking, API-key redaction in logs
Loader, cleaner, chunker	22	1-based pages, empty/invalid PDFs, header removal, fragment re-joining, bullets, overlap, metadata
Retrieval	13	All three modes, subject/document filters, BM25 cache invalidation, embedding-model mismatch
RAG chain and memory	33	Cited answers, refusal without an LLM call, follow-up rewriting safeguards, streaming
API	12	Upload → ask → follow-up → feedback → reindex → delete; every error code; path-traversal filenames
Frontend	8	Page rendering, streamed and non-streamed answers, backend down, interrupted answers
Evaluation code	7	Hit@k, MRR, judge-score parsing, dataset validation

Evaluation results

Evaluated on three uploaded course PDFs (magnetic circuits, DC/AC machines, communication skills; 53 pages) with 30 questions: 26 answerable, worded as an exam would ask them, and 4 that should be refused, including two near-domain traps (transformer EMF equation; stepper vs servo motor). Embeddings: all-MiniLM-L6-v2; answers and judging: llama3.2:3b on an RTX 4050 laptop GPU.

Metric	Result	Target
Hit@5 (correct page in top 5)	1.00 (all modes, chunk size 500 and 1000)	≥ 0.80
MRR (rank of first correct page)	0.95 (MMR, 1000); range 0.92–0.96	≥ 0.60
Refusal of out-of-scope questions	4 / 4 = 100% (threshold 1, LLM 3)	≥ 90%
Answer relevance (LLM judge, 1–5)	4.23	≥ 4.0
Faithfulness (LLM judge, 1–5)	3.46 – 3.69	≥ 4.0 (not met)
Latency per answer (local 3B model)	5.4 s	< 20 s

Performance

Situation	Time	Notes
First answer after starting the backend	5.6 s (was ~38 s)	Embedding model and LLM preloaded in the background at startup
Typical answer	~5 s	Generation-bound: ~68 tokens/s on the RTX 4050; reducing top-k 5 → 3 saved only 0.2 s
Follow-up question	+ ~3 s	One extra LLM call to rewrite the follow-up; skipped for complete questions
Out-of-scope question	~0.03 s	Refused by the score gate before any LLM call
Idle for 5+ minutes	No reload	Ollama keeps the model in GPU memory for 30 minutes (configurable)

Observations

- Refusal needs both layers: the similarity threshold alone stopped only 1 of 4 out-of-scope questions because the traps share vocabulary with the notes; the grounded prompt made the LLM refuse the remaining three.

- Faithfulness is below target and hard to measure with a 3B judge: between two runs 12 of 26 scores changed, some by 3 points, so differences under about ± 0.3 are noise. A stronger judge model would give steadier numbers.
- The weakest answers come from comparison tables, whose structure is lost in PDF text extraction; structure-aware table extraction and OCR for scanned notes are the main future improvements.

7. Future Scope

The planned improvements below are listed in the author's order of priority. The current design keeps each of them contained: the UI talks to the backend only over HTTP, the language model is chosen by a single setting, and new text sources can feed the existing cleaning and chunking steps.

7.1 Improved user interface

- A more polished design: a custom theme with light and dark modes, and a layout that also works on phones.
- A built-in PDF viewer: clicking a citation such as [1] opens the cited page with the passage highlighted.
- Drag-and-drop upload with a progress bar for each file, and a re-index button in the library.
- Exporting a conversation as PDF or Markdown revision notes.
- Longer term, a React or Next.js frontend. Because the current UI uses only the REST and streaming API, a new frontend needs no backend changes.

7.2 OCR support for scanned and handwritten notes

- Today a scanned PDF has no text layer and is rejected with `EMPTY_PDF`.
- Planned: when a page has no extractable text, render it as an image and run OCR (Tesseract, which PyMuPDF can call directly, or PaddleOCR / EasyOCR for better handwriting), then pass the text through the same cleaner and chunker.
- Mark OCR-derived chunks in their metadata and show this on the source card, since OCR text can contain errors.
- Later, a vision-capable model could read diagrams and handwritten formulas.

7.3 Gemini or Grok instead of OpenAI

- The answer model is already selected by one setting (`LLM_PROVIDER = openai | ollama`) in `app/llm/factory.py`, so adding a provider is a small, isolated change.
- Google Gemini through the `langchain-google-genai` package (`ChatGoogleGenerativeAI`) with `GOOGLE_API_KEY`, and xAI Grok through the `langchain-xai` package (`ChatXAI`) with `XAI_API_KEY`, selected as `LLM_PROVIDER = gemini | grok`.
- API keys stay only in the git-ignored `.env` file, exactly as the OpenAI key is handled today.
- Embeddings can remain the local MiniLM model, so the existing index keeps working and only the answer model changes. Switching embeddings as well would require a re-index, which the model-mismatch check enforces.
- A larger hosted model would also serve as a more reliable judge for the faithfulness evaluation.

7.4 Other future work

- Structure-preserving extraction of tables so comparison questions are answered more faithfully.
- Quiz, flashcard and important-question generation from a selected unit.
- Previous-year paper analysis that maps exam questions to syllabus topics.
- Hindi and English questions using multilingual embeddings.
- User accounts, shared class libraries and cloud deployment on Render (configuration already prepared).